Javascript

A. Ferracani

MICC
Media Integration and Communication Center

http://www.micc.unifi.it/

Javascript

JavaScript is a versatile programming language primarily **used for adding interactivity and dynamic behavior to web pages**.

It allows developers to **manipulate** the content of a webpage, respond to **user actions**, and **communicate with servers** to fetch or send data asynchronously.

Some common use cases of JavaScript include:

- **Client-Side Scripting**: JavaScript is widely used for client-side scripting, enabling developers to create interactive user interfaces, validate form inputs, and handle events like **clicks**, scrolls, and keystrokes without the need for page reloads

Javascript

- **Web Application Development**: JavaScript frameworks and libraries such as React, Angular, and Vue.js are used to build modern web applications with **rich user interfaces** and seamless user experiences.

- **Server-Side Scripting**: With the introduction of Node.js, JavaScript can now be used for server-side scripting as well, allowing developers to build scalable and high-performance web servers and applications.

- **Browser APIs**: JavaScript provides access to various browser APIs (Application Programming Interfaces) that allow developers to manipulate the browser's behavior, interact with **multimedia**, access **device sensors** (like geolocation and accelerometer), and perform tasks such as local **storage** and **caching**.

Hello World Example! (http://localhost:8888/html5-ppm/javascript/1-example.html)

```
// JavaScript code for a basic "Hello, World!" example

// Function to display a greeting message
function greet() {
    // Get the element with id "output" from the HTML document
    var outputElement = document.getElementById("output");

    // Check if the element exists
    if (outputElement) {
        // Update the content of the element with a greeting
message
        outputElement.innerText = "Hello, World!";
    }
}

// Call the greet function when the page finishes loading
window.onload = greet;
```

Javascript

- **Origins (1995-1997)**: JavaScript was created by Brendan Eich for Netscape Communications Corporation. It was first introduced in the Netscape **Navigator 2.0 browser in December 1995**. Initially, the language was named Mocha but was later renamed JavaScript to capitalize on the growing popularity of Java

- **Standardization (1997-2009)**: JavaScript became an international standard with the introduction of **ECMAScript in 1997**. ECMAScript is the standardized specification that defines the JavaScript programming language. Over the years, **several versions of ECMAScript** have been released, each introducing new features and improvements to the language.

Javascript

- **The Era of Modern Frameworks (2010-present)**: In recent years, modern JavaScript frameworks like AngularJS, **React**, and Vue.js have revolutionized web development, enabling the creation of complex and responsive user interfaces.

  These frameworks support the concept of **Single Page Applications (SPAs)**, where the entire application is loaded only once, and content is updated dynamically without reloading the page.

Javascript

- **ECMAScript 6 and Beyond (2015-present):** ECMAScript 6, also known as ES6 or ES2015, was one of the most significant versions of JavaScript, introducing many new features such as classes, arrow functions, constants, template strings, modules, and much more.

  Since then, new versions of ECMAScript have been released every year, constantly introducing new features and improvements to the language.

Javascript

- **Ajax and the Framework Era (2005-2010):** The asynchronous programming technique Ajax (**Asynchronous JavaScript and XML**) gained popularity, allowing web developers to create more dynamic and interactive applications.

  During this period, various JavaScript frameworks emerged, such as **jQuery**, Prototype, Dojo, and many others, to simplify web development.

Javascript

You can **include JavaScript in an HTML document** in several ways. Here are some of the most common methods

Method 1: <script> Tag in the <head> Element

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
    <title>Document</title>
    <!-- Include the JavaScript script -->
    <script src="script.js"></script>
</head>
<body>
    <!-- Page content -->
</body>
</html>
```

Javascript

You can also include JavaScript at the end of the HTML document body, just before the closing </body> tag. This can improve page loading performance as the script runs after the rest of the page has been loaded.

Method 2: <script> Tag at the End of the Body

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>Document</title>
</head>
<body>
    <!-- Page content -->

    <!-- Include the JavaScript script at the end of the body -->
    <script src="script.js"></script>
</body>
</html>
```

Javascript

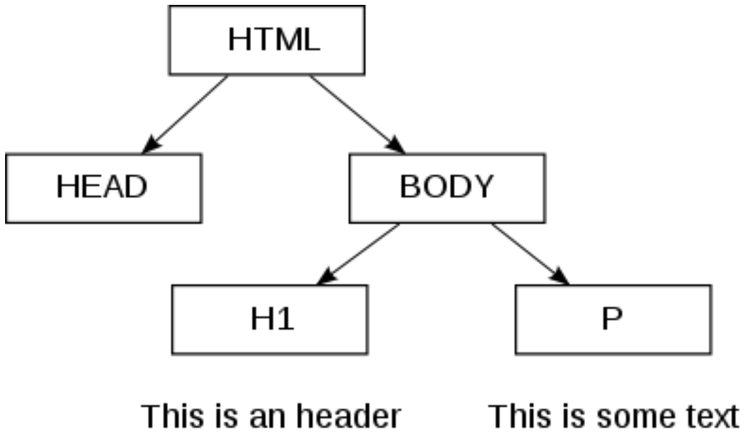
You can also include JavaScript directly inside a <script> tag in the HTML. This is known as "inline JavaScript"

Method 3: Page or inline JavaScript (http://localhost:8888/html5-ppm/javascript/2-example.html)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>Document</title>
</head>
<body>
    <!-- Page content -->
    <!-- Page JavaScript –>
    <button onclick=“alert(‘clicked’)”>Click me!</button>
    <script>
        alert("This is an alert generated by inline JavaScript!");
    </script>
</body>
</html>
```

Some examples: dom manipulations

- Example 1 - getElementById
- Example 2 - inline javascript
- Example 3 - page javascript
- Example 4 - change attribute value
- Example 5 - event handler
- Example 6 - dynamic event creation
- Example 7 - keyboard event
- Example 8 - dragging
- Example 9 - drawing




```
|-> Document
  |-> Element (<html>)
    |-> Element (<body>)
      |-> Element (<div>)
        |-> text node
        |-> Anchor
          |-> text node
      |-> Form
          |-> Text-box
          |-> Text Area
          |-> Radio Button
          |-> Check Box
          |-> Select
          |-> Button
```

Javascript

Before the introduction of ECMAScript 6 (ES6), JavaScript had some fundamental characteristics in its constructs. features:

Functions: they can be defined explicitly using the function keyword. Before ES6, functions could also be defined as function expressions.

```
// Explicitly defined function
function greet(name) {
    return "Hello, " + name + "!";
}

// Function as a function expression
var greet = function(name) {
    return "Hello, " + name + "!";
};
```

Javascript

Variables: In JavaScript, variables can be declared using **var**, which has function scope, and are typically declared at the beginning of a function or script

```
var x = 5;
```

Conditionals: JavaScript supports conditional statements if, else if, and else to execute code based on conditions.

```
var x = 10;

if (x > 5) {
    console.log("x is greater than 5");
} else {
    console.log("x is not greater than 5");
}
```

Javascript

Loops: JavaScript supports for, while, and do...while loops to iterate over a series of statements.

```
// For loop

for (var i = 0; i < 5; i++) {
    console.log(i);
}

// While loop
var i = 0;
do while (i < 5) {
    console.log(i);
    i++;
}
```

Javascript

Objects and Arrays: JavaScript is prototype-based, so objects can be created using the **{}** literal syntax, and arrays can be created using the **[]** syntax.

```
// Object
var person = {
    name: "John",
    age: 30
};

// Array
var numbers = [1, 2, 3, 4, 5];
```

Javascript

Classes: in plain old javascript there are **no classes**. Function are classes - http://localhost:8888/ajax_course_2017/examples/4_oop/oop_5.html

```
// constructor
function Table (rows, columns) {
   //instance properties
   this.rows = rows;
   this.columns = columns;
   // getCellCount method
   this.getCellCount = getCellCount;
}

// function to return back cell num
function getCellCount() {
   return this.rows * this.columns;
}

var t = new Table(3,5);
var cellCount = t.getCellCount();
document.write("I just created a table of " + cellCount + " cells");
```

Javascript

Prototyping: old javascript is a prototyping language - http://localhost:8888/ajax_course_2017/examples/4_oop/oop_6.html

```
// class Table
function Table (rows, columns) {
   // instance properties
   this.rows = rows;
   this.columns = columns;
}

// Table.getCellCount to return the number of cells
Table.prototype.getCellCount = function() {
   return this.rows * this.columns;
};

var t1 = new Table(2,3);
var t2 = new Table(3,5);

// first table (6 cells)
document.write("first table cells: " + t1.getCellCount());

// second table cells (15 cells)
document.write("second table cells" + t2.getCellCount());
```

Javascript

Static methods and properties: http://localhost:8888/ajax_course_2017/examples/4_oop/oop_7.html

```
// class Table
function Table (rows, columns) {
    // instance properties
    this.rows = rows;
    this.columns = columns;
}

// Table.getCellCount
Table.prototype.getCellCount = function() {
    return this.rows * this.columns;
};

// static property
Table.SQUARESIZE = 2;
// static method
Table.getSquareTable = function() {
    return new Table(Table.SQUARESIZE, Table.SQUARESIZE);
}

var t3 = Table.getSquareTable();

document.write(“squared table cell count " + t3.getCellCount() + "cells");
```

Javascript

Public/Private properties: do not exist in javascript but you can simulate them using closures - http://localhost:8888/ajax_course_2017/examples/4_oop/oop_8.html

```
function Table (rows, columns) {
    // save parameter values to local variables
    var _rows = rows;
    var _columns = columns;
    // return the number of table cells
    this.getCellCount = function() {
        return _rows * _columns;
    };
}

var t = new Table(3,5);

document.write("Your table has " + t._rows + " rows" +
" and " + t._columns + " columns<br />");

document.write("The table has " + t.getCellCount() + " cells");
```

Javascript

The **Browser Object Model (BOM)** in JavaScript represents the browser window and its components. While the **Document Object Model (DOM)** represents the structure of the web page as a tree of nodes, the BOM provides objects and methods to interact with the browser itself, the user's system, and the environment in which the browser is running

Window Object: The **window** object is the top-level object in the BOM hierarchy. It represents the browser window itself and serves as the global object for JavaScript code running in the browser.

The **window** object provides properties and methods for controlling the browser window, such as resizing the window, navigating to URLs, opening new windows, and interacting with the browser's location bar.

Example: http://localhost:8888/eclipse_projects/ajax_course_2025/javascript_1/bom_window.html

Javascript

Screen Object: The screen object provides information about the user's screen or display. It includes properties like **screen.width** and **screen.height** (provides the width and height of the screen in pixels), **screen.availWidth** and **screen.availHeight** (provides the width and height of the screen excluding taskbars and other system elements), and more.

Example: http://localhost:8888/eclipse_projects/ajax_course_2025/javascript_1/bom_screen.html

Navigator Object: The navigator object provides information about the browser and the user's system. It includes properties like **navigator.userAgent** (provides information about the user agent string), **navigator.platform** (provides information about the operating system platform), and **navigator.language** (provides information about the user's preferred language).

Example: http://localhost:8888/eclipse_projects/ajax_course_2025/javascript_1/bom_navigator.html

Javascript

History Object: The history object represents the user's browsing history in the current window. It provides methods like **history.back()** (navigates back to the previous page in the history), **history.forward()** (navigates forward in the history), and **history.go()** (navigates to a specific page in the history).

Example: http://localhost:8888/eclipse_projects/ajax_course_2025/javascript_1/bom_history.html

Location Object: The location object represents the URL of the current web page loaded in the browser window. It provides information about the URL's components such as **location.href** (the complete URL), **location.hostname** (the hostname part of the URL), **location.pathname** (the path part of the URL), and more. It also allows navigation to different URLs through methods like **location.assign()** and **location.reload()**.

Example: http://localhost:8888/eclipse_projects/ajax_course_2025/javascript_1/bom_location.html

Javascript

Document Object: While the Document Object Model (DOM) is not strictly part of the BOM, it is closely related and represents the structure of the web page document loaded in the browser. The **document** object provides methods and properties to interact with the elements and content of the web page, such as accessing and modifying HTML elements, handling events, and more.

Javascript

References

- Javascript intro: http://www.w3schools.com/jsref/default.asp
- JSON samples: https://www.w3schools.com/js/js_json_intro.asp

Ajax

- **AJAX** stands for **Asynchronous JavaScript and XML**: is a programming technique used to create interactive web applications that can **exchange data with a web server in the background without needing to reload the entire page**

- The term "AJAX" was coined by **Jesse James Garrett** in a blog post titled "Ajax: A New Approach to Web Applications," published in February 2005

- The key idea behind AJAX is to leverage the **XMLHttpRequest** (XHR) object, introduced by Microsoft in Internet Explorer 5, to perform asynchronous data exchange between the client-side and server-side of web applications.

Ajax

- **AJAX** stands for **Asynchronous JavaScript and XML**: a programming technique used to create interactive web applications that can **exchange data with a web server in the background without needing to reload the entire page**

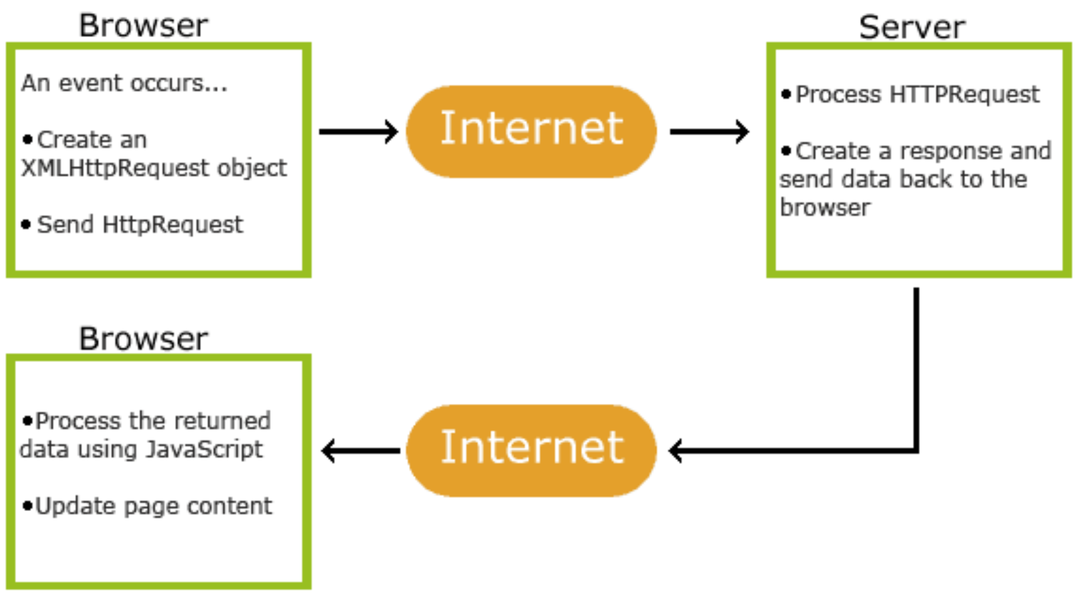
- **XMLHttpRequest (XHR) Object:** This is a JavaScript object that provides the means to send asynchronous HTTP requests to the server and handle the responses

- **Event Handling**: AJAX utilizes JavaScript events to handle the asynchronous nature of data exchange. Events such as **onreadystatechange** are used to track the progress of requests and execute appropriate code when responses are received

- **XML or JSON:** While the "X" in AJAX stands for XML, data exchange is not limited to XML. **JSON (JavaScript Object Notation)** has become more popular due to its lightweight and easily parseable nature

- **DOM Manipulation**: AJAX allows developers to update portions of a web page dynamically without needing to reload the entire page. This is achieved through Document Object Model (DOM) manipulation

Ajax



References

- Ajax sample: https://www.w3schools.com/xml/xml_http.asp

Example 1: Basic XMLHttpRequest: https://onecompiler.com/html/429pmphx3

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>AJAX Example with Button Click</title>
</head>
<body>
    <div id="result"></div>
    <button id="loadDataButton">Load Data</button>

    <script>
        // Function to make AJAX request
        function fetchData() {
            // Create an XMLHttpRequest object
            var xhr = new XMLHttpRequest();
            // Configure the request
            xhr.open('GET', 'https://jsonplaceholder.typicode.com/posts/1', true);
            // Define the callback function to handle the response
            xhr.onload = function() {
                if (xhr.status >= 200 && xhr.status < 300) {
                    // Parse the JSON response
                    var data = JSON.parse(xhr.responseText);
                    // Update the DOM with the response
                    document.getElementById('result').textContent = data.title;
                } else {
                    console.error('Request failed with status:', xhr.status);
                }
            };
            // Send the request
            xhr.send();
        }
        // Add click event listener to the button
        document.getElementById('loadDataButton').addEventListener('click', fetchData);
    </script>
</body>
</html>
```

jQuery

Is a JS library for DOM manipulation: https://onecompiler.com/html/429vu9f4h

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>AJAX Example with Button Click</title>
    <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
</head>
<body>
    <div id="result"></div>
    <button id="loadDataButton">Load Data</button>

    <script>
        $(document).ready(function() {
            // Function to make AJAX request
            function fetchData() {
                // Make AJAX request using jQuery
                $.get('https://jsonplaceholder.typicode.com/posts/1', function(data, status) {
                    if (status === 'success') {
                        // Update the DOM with the response
                        $('#result').text(data.title);
                    } else {
                        console.error('Request failed with status:', status);
                    }
                });
            }
            // Add click event listener to the button
            $('#loadDataButton').click(fetchData);
        });
    </script>
</body>
</html>
```

References

- jQuery examples: https://www.w3schools.com/jquery/jquery_examples.asp
- jQuery selectors: https://www.w3schools.com/jquery/jquery_ref_selectors.asp
- jQuery course: http://localhost:8888/jquery_course/
- jQuery slider plugin: http://localhost:8888/jquery-slider/
- jQuery to do list plugin: http://localhost:8888/to-do-list-2018/to-do-list-completed/

Javascript

ES6 or the ECMAScript 2015 is the 6th and major edition of the ECMAScript language specification standard.

It defines the standard for the implementation of JavaScript and it has become much more popular than the previous edition

New features

- let and const Keywords
- Arrow Functions
- Multi-line Strings
- Default Parameters
- Template Literals
- Destructuring Assignment
- Enhanced Object Literals
- Promises
- Fetch API
- Classes
- Modules

let and const Keywords

previously declared with **var** that could be used before declaration and could have global or function scope

let and **const**: cannot be used before declaration; they have block scope

```
let i = 10;
console.log(i);    //Output 10

const PI = 3.14;
console.log(PI);   //Output 3.14
```

let and const Keywords

```
if (true) {
  var a = 10;
  let b = 20;
}

console.log(a);
// Outputs: 10 (accessible outside the
block)

console.log(b);
// ReferenceError: b is not defined
outside the block
```

Var, let and const Keywords: Hoisting and the Temporal Dead Zone (TDZ)

var: declarations using var are hoisted to the top of their function or global scope. However, they are initialized with **undefined** until the actual assignment is reached

let: declarations using let are also hoisted, but they are not initialized. Accessing them before their declaration results in a ReferenceError due to the Temporal Dead Zone.

```
console.log(x); // Outputs:
undefined
var x = 5;
```

```
console.log(y); //
ReferenceError: Cannot
access 'y' before
initialization
let y = 10;
```

Var, let and const Keywords: redefinition

var: allows the same variable to be redeclared within the same scope, which can lead to potential bugs

let: does not allow the redeclaration of the same variable within the same block, which helps prevent accidental overwriting

```
var z = 1;
var z = 2; // No error, z
is now 2
console.log(z); // Outputs:
2
```

```
let w = 1;
let w = 2;
// SyntaxError: Identifier
'w' has already been
declared
```

Arrow functions

ES6 provides a more concise way

es6

```
// Arrow function
let sumOfTwoNumbers = (a, b)
=> a + b;

console.log(sumOfTwoNumbers(10
, 20));
// Output 30
```

pre-es6

```
//function
function sumOfTwoNumbers(a, b) {
 return a + b;
}

console.log(sumOfTwoNumbers(10,
20));
// Output 30
```

Multi-line strings

By using back ticks `….`

es6

```
let greeting =
`Hello World,
Greetings to all,
Keep Learning and
Practicing!`
```

pre-es6

```
let greeting =
‘Hello World,’ +
‘Greetings to all,’ +
‘ Keep Learning and
Practicing!’
```

Default parameters

Can be used

es6

```
//ES6
let calculateArea =
function(height = 100,
width = 50) {
    // logic
}
```

pre-es6

```
//ES5
var calculateArea =
function(height, width) {
   height =  height || 50;
   width = width || 80;
   // logic
}
```

Template literals

Syntax for using the string template is ${PARAMETER} and is used inside of the back-ticked string

```
let name = `My name is ${firstName} ${lastName}`
```

Destructuring assignement (https://onecompiler.com/javascript/43de9ht5c)

makes it easy to extract values from arrays, or properties from objects into distinct variables

es6

```
//Array Destructuring
let fruits = ["Apple",
"Banana"];
let [a, b] = fruits; // Array
destructuring assignment
console.log(a, b);

//Object Destructuring
let person = {name: "Peter",
age: 28};
let {name, age} = person; //
Object destructuring assignment
console.log(name, age);
```

pre-es6

```
//Old way
let fruits = ["Apple", "Banana"];
let a = fruits[0];
let b = fruits[1];
console.log(a, b);

//Old way
let person = {name: "Peter", age:
28};
let name = person.name;
let age = person.age;
console.log(name, age);
```

Enhanced object literals (https://onecompiler.com/javascript/43de9sphc)

make it easy to quickly create objects with properties inside the curly braces

es6

```
function getMobile(manufacturer,
model, year) {
   return {
      manufacturer,
      model,
      year
   }
}

getMobile("Samsung", "Galaxy",
"2020");
```

pre-es6

```
function getMobile(manufacturer,
model, year) {
   return {
      "manufacturer": manufacturer,
      "model": model,
      "year": year
   }
}

getMobile("Samsung", "Galaxy",
"2020");
```

Promises (https://onecompiler.com/javascript/43de9yxqp)

Promises are used for asynchronous execution. We can use promise with the arrow function as demonstrated below

es6: creation and consumption

```
let asyncCall =  new Promise((resolve, reject) => {
   // do something
   resolve('Operation completed successfully');
}).then((result)=> {
   console.log(‘OK!');
}).catch((error) => {
  console.error(error); // Output: Error: Operation
failed
});
```

Javascript

- new Promise((resolve, reject) => { ... }): viene creata una nuova **Promise**. La funzione passata al costruttore riceve due callback: **resolve**: da chiamare quando l'operazione asincrona va a buon fine. **reject**: da chiamare se l'operazione fallisce

- resolve('Operation completed successfully'): l'operazione è simulata e si risolve immediatamente, restituendo la stringa 'Operation completed successfully'

- .then((result) => { ... }): il metodo .**then()** viene usato per definire una funzione che verrà eseguita quando la Promise viene risolta

- result: è il valore passato a **resolve()**, in questo caso la stringa 'Operation completed successfully'

Example of chaining Promises (https://onecompiler.com/javascript/43dear3rq)

```
function fetchData() {
  return new Promise((resolve, reject) => {
    // Simulate fetching data from an API
    setTimeout(() => {
      let data = { id: 1, name: 'John' };
      resolve(data);
    }, 1000);
  });
}

function processData(data) {
  return new Promise((resolve, reject) => {
    // Simulate processing data
    setTimeout(() => {
      data.age = 30;
      resolve(data);
    }, 1000);
  });
}

function saveData(data) {
  return new Promise((resolve, reject) => {
    // Simulate saving data
    setTimeout(() => {
      console.log('Data saved:', data);
      resolve('Data saved successfully');
    }, 1000);
  });
}
```

```
// Chaining promises
fetchData()
  .then(processData)
  .then(saveData)
  .then((message) => {

console.log(message); //
Output: Data saved
successfully
  })
  .catch((error) => {
  console.error('Error:',
error);
  });
```

Example of chaining with await / async (http://localhost:8888/eclipse_projects/ajax_course_2025/examples/fetch-async-await-example.html)

```
function fetchData() {
  // Implicitly returns a promise
  return fetch('https://
jsonplaceholder.typicode.com/posts')
    .then(response => response.json());
}

function processData(data) {
  return new Promise((resolve, reject) => {
    // Simulate processing data
    setTimeout(() => {
      data.age = 30;
      resolve(data);
    }, 1000);
  });
}

function saveData(data) {
  return new Promise((resolve, reject) => {
    // Simulate saving data
    setTimeout(() => {
      console.log('Data saved:', data);
      resolve('Data saved successfully');
    }, 1000);
  });
}
```

```
// Chaining promises with async/
await
async function example() {
  try {
    let result1 = await fetchData();
// Wait for fetchData promise to
resolve
    let result2 = await
processData(result1); // Wait for
processData promise to resolve
    let result3 = await
saveData(result2); // Wait for
saveData promise to resolve
    console.log(result3); // Output:
Data saved successfully
  } catch (error) {
    console.error('Error:', error);
  }
}

example();
```

ES8: await / async

- async and await are keywords in JavaScript used to work with **asynchronous code** in a more synchronous-like manner, making asynchronous code easier to read and write

- **async**: The async keyword is used to declare an asynchronous function. An async function **always returns a promise**, and the value of the promise will be whatever the async function returns.

- **await**: The await keyword is used inside an async function to pause the execution of the function until a promise is resolved. **It can only be used within an async function**. When await is used, it waits for the promise to resolve, and then it returns the resolved value.

Fetch API

The Fetch API is a modern alternative to XMLHttpRequest for making asynchronous HTTP requests in JavaScript.

Chaining .then(): After calling fetch(), you can chain **.then()** methods to the returned Promise. Each .then() method takes a callback function that will be executed when the Promise is resolved (i.e., when the network request completes successfully).

es6

```
fetch(url, options)
    .then(response => {
        // Handle response
    })
    .catch(error => {
        // Handle error
    });
```

Example 2: fetch API - ES6: https://onecompiler.com/html/429pmzm69

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>AJAX Example with Fetch API and Button Click</title>
</head>
<body>
    <div id="result"></div>
    <button id="loadDataButton">Load Data</button>

    <script>
        // Function to fetch data using Fetch API
        function fetchData() {
            // Make a GET request using Fetch API
            fetch('https://jsonplaceholder.typicode.com/posts/1')
                .then(response => {
                    if (!response.ok) {
                        throw new Error('Network response was not ok');
                    }
                    return response.json();
                })
                .then(data => {
                    // Update the DOM with the response
                    document.getElementById('result').textContent = data.title;
                })
                .catch(error => {
                    console.error('There was a problem with the fetch operation:', error);
                });
        }

        // Add click event listener to the button
        document.getElementById('loadDataButton').addEventListener('click', fetchData);
    </script>
</body>
</html>
```

Ajax

Example 3: fetch with await async: https://onecompiler.com/html/42a33syds

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>AJAX Example with Fetch API and Button Click</title>
</head>
<body>
    <div id="result"></div>
    <button id="loadDataButton">Load Data</button>

    <script>
        // Function to fetch data using Fetch API
        async function fetchData() {
            try {
                // Make a GET request using Fetch API
                const response = await fetch('https://jsonplaceholder.typicode.com/posts/1');
                if (!response.ok) {
                    throw new Error('Network response was not ok');
                }
                const data = await response.json();
                // Update the DOM with the response
                document.getElementById('result').textContent = data.title;
            } catch (error) {
                console.error('There was a problem with the fetch operation:', error);
            }
        }

        // Add click event listener to the button
        document.getElementById('loadDataButton').addEventListener('click', fetchData);
    </script>
</body>
</html>
```

Classes (https://onecompiler.com/javascript/43dedmw2t)

Previously, classes never existed in JavaScript

es6

```
class UserProfile {

   constructor(firstName, lastName) {
      this.firstName = firstName;
      this.lastName = lastName;
   }

   getName() {
     console.log(`The Full-Name is ${this.firstName} ${this.lastName}
`);
   }
}

let obj = new UserProfile('John', 'Smith');
obj.getName(); // output: The Full-Name is John Smith
```

Inheriting (https://onecompiler.com/javascript/43deev38t)

es6

```
class AdminProfile extends UserProfile {
  constructor(firstName, lastName, role) {
    super(firstName, lastName); // Inizializza firstName e lastName
    this.role = role;           // Inizializza l'attributo specifico
  }

  // Metodo aggiuntivo per mostrare il ruolo
  getRole() {
    console.log(`Role: ${this.role}`);
  }

  // Override del metodo getName per personalizzare il messaggio
  getName() {
    console.log(`Admin Full-Name: ${this.firstName} ${this.lastName}`);
  }
}

// Esempio d'uso:
let admin = new AdminProfile('Alice', 'Wonderland', 'Administrator');
admin.getName(); // Output: Admin Full-Name: Alice Wonderland
admin.getRole(); // Output: Role: Administrator
```

Classes: instance fileds

- **getter**: allow controlled access to properties as if they were virtual properties
- **setter**: enable modification of properties with added logic for validation or transformation
- the **underscore** prefix (e.g., _firstName) is a convention to indicate that a property is “private” before the introduction of official private fields.

Classes: getter e setter (https://onecompiler.com/javascript/43dnq7qkv)

es6

```
class Person {
  constructor(name, surname) {
    this._nome = name;
    this._cognome = surname;
  }

  // Getter for complete name
  get completeName() {
    return `${this._name} ${this._surname}`;
  }

  // Setter for name and surname
  set completeName(value) {
    const parts = value.split(' ');
    this._name = parts[0];
    this._surname = parts[1];
  }
}

let p = new Person('Mario', 'Rossi');
console.log(p.completeName); // Output: Mario Rossi

p.completeName = 'Luigi Bianchi';
console.log(p.completeName); // Output: Luigi Bianchi
```

Classes: instance fields

- Belong to each instance of the class
- Can be declared directly in the class (from ES2020 onward)

```
class Car {
  make = 'Toyota'; // Public instance field

  constructor(model) {
    this.model = model;
  }
}

let car1 = new Car('Corolla');
console.log(car1.make); // Output: Toyota
```

Classes: static fields

- belong to the class itself rather than any instance
- useful for utility methods or constants.

```
class MathUtility {
  static sum(a, b) {
    return a + b;
  }

  multiply(a, b) {
    return a * b;
  }
}

console.log(MathUtility.sum(5, 10)); // Output: 15
let util = new MathUtility();
console.log(util.multiply(5, 10)); // Output: 50
```

Classes: public vs private

- **Public Fields**: accessible from anywhere and declared without any special prefix
- **Private Fields**: declared using the # prefix and are only accessible within the class
- **Advantages of Private Fields**:
  - encapsulation: Protects the internal state of the class
  - control: Allows managing access through methods (like getters and setters) to prevent unwanted modifications

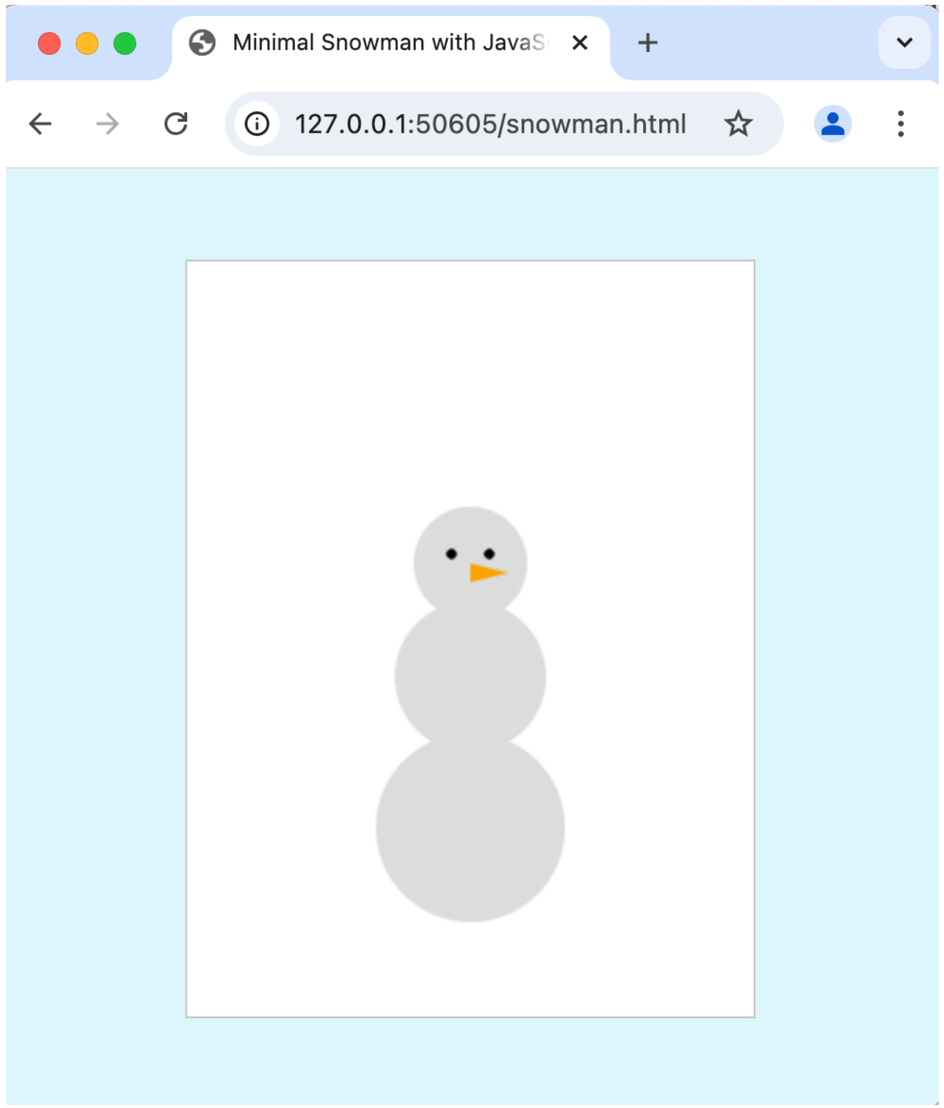
```
class Rectangle {
  #height = 0;
  #width;

  constructor(height, width) {
    this.#height = height;
    this.#width = width;
  }

  getArea() {
    return this.#height *
this.#width;
  }
}

let rect = new Rectangle(10, 5);
console.log(rect.getArea());
// Output: 50
console.log(rect.#height);
// ERROR: Direct access to a private
field is not allowed
```

Javascript



http://localhost:8888/eclipse_projects/ajax_course_2025/examples/snowman.html